

Remove misuses of list-initialization from Clause 24

Document #: P2367R0
Date: 2021-04-29
Project: Programming Language C++
Audience: LWG
Reply-to: Tim Song
<t.canens.cpp@gmail.com>

§ 1 Abstract

This paper provides wording for [\[LWG3524\]](#) and resolves related issues caused by the erroneous use of list-initialization in ranges wording.

§ 2 Discussion

As discussed in [\[LWG3524\]](#), the use of list-initialization in the ranges specification implies ordering guarantees that are unintended and unimplementable in ordinary C++, as well as narrowing checks that are unnecessary and sometimes unimplementable.

Other issues have also been reported against the misuse of list-initialization in ranges wording. See [\[subrange\]](#) and [\[take.view\]](#).

The approach taken in this paper is to remove uses of list-initialization that are actively harmful, as well as any neighboring uses to maintain some level of local consistency. I have not attempted to remove all uses of list-initialization in the normative text of Clause 24.

During the drafting of this paper, two issues were discovered with `views::single`:

- The implicit deduction guides fail to remove cv-qualification for const rvalues or decay array/function types. This is inconsistent with range-v3 and appears to be an oversight.
- `views::single(a_single_view)` unexpectedly copies instead of wraps, similar to [\[LWG3474\]](#).

The proposed wording below fixes these issues as well.

§ 3 Wording

This wording is relative to [\[N4885\]](#).

1. Edit 24.5.4.2 [\[range.subrange.ctor\]](#) p6 as indicated:

```
template<not-same-as<subrange> R>
    requires borrowed_range<R> &&
           convertible-to-non-slicing<iterator_t<R>, I> &&
           convertible_to<sentinel_t<R>, S>
constexpr subrange(R&& r) requires (!StoreSize || sized_range<R>);
```

6 Effects: Equivalent to:

(6.1) — If *StoreSize* is *true*, `subrange{r, ranges::size(r)}`.

(6.2) — Otherwise, `subrange{ranges::begin(r), ranges::end(r)}`.

2. Edit 24.6.3.1 [\[range.single.overview\]](#) p2 as indicated:

- 2 The name `views::single` denotes a customization point object (16.3.3.3.6 [\[customization.point.object\]](#)). Given a subexpression *E*, the expression `views::single(E)` is expression-equivalent to `single_view<decay_t<decltype((E))>>{E}`.

3. Add a deduction guide to 24.6.3.2 [\[range.single.view\]](#):

```
namespace std::ranges {
    template<copy_constructible T>
        requires is_object_v<T>
        class single_view : public view_interface<single_view<T>> {
            // [...]
        };

+   template<class T>
+       single_view(T) -> single_view<T>;
}
```

4. Edit 24.6.4.1 [\[range.iota.overview\]](#) p2 as indicated:

- 2 The name `views::iota` denotes a customization point object (16.3.3.3.6 [\[customization.point.object\]](#)). Given subexpressions *E* and *F*, the expressions `views::iota(E)` and `views::iota(E, F)` are expression-equivalent to `iota_view{E}` and `iota_view{E, F}`, respectively.

5. Edit 24.7.5.1 [\[range.filter.overview\]](#) p2 as indicated:

- 2 The name `views::filter` denotes a range adaptor object (24.7.2 [\[range.adaptor.object\]](#)). Given subexpressions *E* and *P*, the expression `views::filter(E, P)` is expression-equivalent to `filter_view{E, P}`.

6. Edit 24.7.6.1 [\[range.transform.overview\]](#) p2 as indicated:

- 2 The name `views::transform` denotes a range adaptor object (24.7.2 [\[range.adaptor.object\]](#)). Given subexpressions *E* and *F*, the expression `views::transform(E, F)` is expression-equivalent to `transform_view{E, F}`.

7. Edit 24.7.7.1 [\[range.take.overview\]](#) p2 as indicated:

2 The name `views::take` denotes a range adaptor object (24.7.2 [\[range.adaptor.object\]](#)). Let `E` and `F` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::take(E, F)` is ill-formed. Otherwise, the expression `views::take(E, F)` is expression-equivalent to:

- (2.1) — If `T` is a specialization of `ranges::empty_view` (24.6.2.2 [\[range.empty.view\]](#)), then `((void) F, decay-copy(E))`, except that the evaluations of `E` and `F` are **indeterminately sequenced**.
- (2.2) — Otherwise, if `T` models `random_access_range` and `sized_range` and is
 - (2.2.1) — [...]

then `T{ranges::begin(E), ranges::begin(E) + min<D>(ranges::size(E), F)}`, except that `E` is evaluated only once.
- (2.3) — Otherwise, `ranges::take_view{E, F}`.

8. Edit 24.7.7.2 [\[range.take.view\]](#) as indicated:

```
namespace std::ranges {
    template<view V>
    class take_view : public view_interface<take_view<V>> {

        // [...]

    constexpr auto begin() requires (!simple-view<V>) {
        if constexpr (sized_range<V>) {
            if constexpr (random_access_range<V>)
                return ranges::begin(base_);
            else {
                auto sz = size();
                return counted_iterator{ranges::begin(base_), sz};
            }
        } else
            return counted_iterator{ranges::begin(base_), count};
    }

    constexpr auto begin() const requires range<const V> {
        if constexpr (sized_range<const V>) {
            if constexpr (random_access_range<const V>)
                return ranges::begin(base_);
            else {
                auto sz = size();
                return counted_iterator{ranges::begin(base_), sz};
            }
        } else
            return counted_iterator{ranges::begin(base_), count};
    }

    // [...]
};
```

```
template<class R>
    take_view(R&&, range_difference_t<R>)
        -> take_view<views::all_t<R>>;
}
```

9. Edit 24.7.8.1 [\[range.take.while.overview\]](#) p2 as indicated:

- 2 The name `views::take_while` denotes a range adaptor object (24.7.2 [\[range.adaptor.object\]](#)). Given subexpressions `E` and `F`, the expression `views::take_while(E, F)` is expression-equivalent to `take_while_view{((E, F))}`.

10. Edit 24.7.9.1 [\[range.drop.overview\]](#) p2 as indicated:

- 2 The name `views::drop` denotes a range adaptor object (24.7.2 [\[range.adaptor.object\]](#)). Let `E` and `F` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::drop(E, F)` is ill-formed. Otherwise, the expression `views::drop(E, F)` is expression-equivalent to:
 - (2.1) — If `T` is a specialization of `ranges::empty_view` (24.6.2.2 [\[range.empty.view\]](#)), then `((void) F, decay-copy(E))`, except that the evaluations of `E` and `F` are **indeterminately sequenced**.
 - (2.2) — Otherwise, if `T` models `random_access_range` and `sized_range` and is
 - (2.2.1) — [...]

then `T{ranges::begin(E) + min<D>(ranges::size(E), F), ranges::end(E)}`, except that `E` is evaluated only once.
 - (2.3) — Otherwise, `ranges::drop_view{((E, F))}`.

11. Edit 24.7.10.1 [\[range.drop.while.overview\]](#) p2 as indicated:

- 2 The name `views::drop_while` denotes a range adaptor object (24.7.2 [\[range.adaptor.object\]](#)). Given subexpressions `E` and `F`, the expression `views::drop_while(E, F)` is expression-equivalent to `drop_while_view{((E, F))}`.

12. Edit 24.7.12.1 [\[range.split.overview\]](#) p2 as indicated:

- 2 The name `views::split` denotes a range adaptor object (24.7.2 [\[range.adaptor.object\]](#)). Given subexpressions `E` and `F`, the expression `views::split(E, F)` is expression-equivalent to `split_view{((E, F))}`.

13. Edit 24.7.12.2 [\[range.split.view\]](#) p2 as indicated:

```
template<input_range R>
    requires constructible_from<V, views::all_t<R>> &&
             constructible_from<Pattern, single_view<range_value_t<R>>>
constexpr split_view(R&& r, range_value_t<R> e);
```

2 *Effects:* Initializes `base_` with `views::all(std::forward<R>(r))`, and `pattern_` with `single_view{std::move(e)} views::single(std::move(e))`.

14. Edit 24.7.13 [\[range.counted\]](#) p2 as indicated:

2 The name `views::counted` denotes a customization point object (16.3.3.3.6 [\[customization.point.object\]](#)). Let `E` and `F` be expressions, let `T` be `decay_t<decltype((E))>`, and let `D` be `iter_difference_t<T>`. If `decltype((F))` does not model `convertible_to<D>`, `views::counted(E, F)` is ill-formed.

[*Note 1:* This case can result in substitution failure when `views::counted(E, F)` appears in the immediate context of a template instantiation. — *end note*]

Otherwise, `views::counted(E, F)` is expression-equivalent to:

- (2.1) — If `T` models `contiguous_iterator`, then `span{to_address(E), static_cast<D>(F)}`.
- (2.2) — Otherwise, if `T` models `random_access_iterator`, then `subrange{E, E + static_cast<D>(F)}`, except that `E` is evaluated only once.
- (2.3) — Otherwise, `subrange{counted_iterator{E, F}, default_sentinel}`.

§ 4 References

[LWG3474] Barry Revzin. Nesting `join_views` is broken because of CTAD.

<https://wg21.link/lwg3474>

[LWG3524] Tim Song. Unimplementable narrowing and evaluation order requirements for range adaptors.

<https://wg21.link/lwg3524>

[N4885] Thomas Köppe. 2021-03-17. Working Draft, Standard for Programming Language C++.

<https://wg21.link/n4885>

[subrange] Jonathan Wakely. 2020. Yet Another Braced Init Bug.

<https://lists.isocpp.org/lib/2020/10/17806.php>

[take.view] Michael Schellenberger Costa. 2021. Fix error in `take_view::begin` by casting to the appropriate size.

<https://github.com/microsoft/STL/pull/1844>